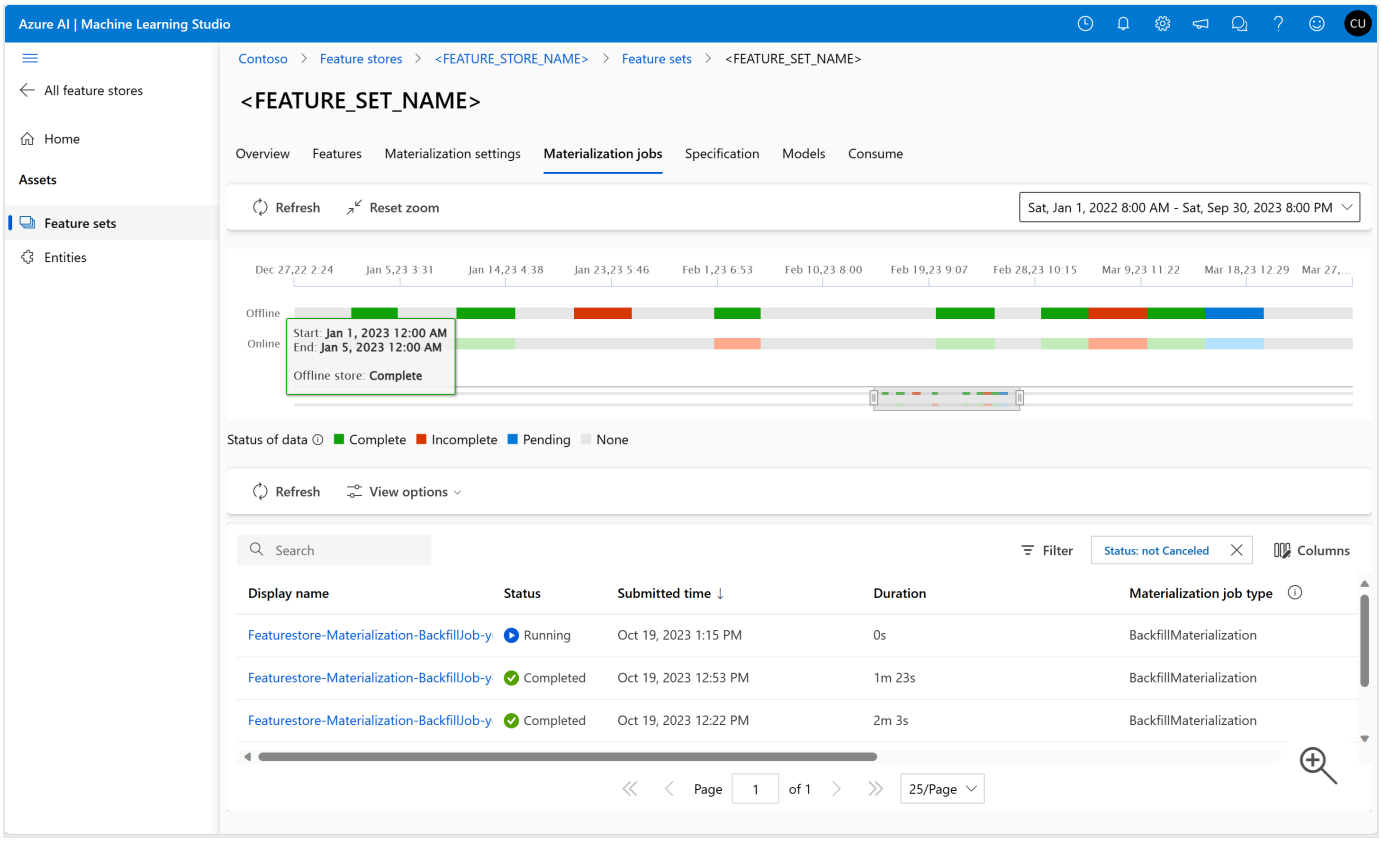
# Feature set materialization concepts

Materialization computes feature values from source data. Start time and end time values define a feature window. A materialization job computes features in this feature window. Materialized feature values are then stored in an online or offline materialization store. After data materialization, all feature queries can then use those values from the materialization store.

Without materialization, a feature set offline query applies the transformations to the source on-the-fly, to compute the features before the query returns the values. This process works well in the prototyping phase. However, for training and inference operations, in a production environment, features should be materialized prior to training or inference. Materialization at that stage provides greater reliability and availability.

## Exploring feature materialization

The **Materialization jobs** UI shows the data materialization status in offline and online materialization stores, and a list of materialization jobs.



In a feature window:

- The time series chart at the top shows the data intervals that fall into the feature window, with the materialization status, for both offline and online stores.
- The job list at the bottom shows all the materialization jobs with processing windows that overlap with the selected feature window.

## Data materialization status and data interval

A data interval is a time window in which the feature set materializes its feature values to one of these statuses:

- Complete (green) - successful data materialization
- Incomplete (red) - one or more canceled or failed materialization jobs for this *data interval*
- Pending (blue) - one or more materialization jobs for this *data interval* are in progress
- None (gray) - no materialization job was submitted for this *data interval*

As materialization jobs run for the feature set, they create or merge data intervals:

- When two data intervals are continuous on the timeline, and they have the same data materialization status, they become one data interval
- In a data interval, when a portion of the feature data is materialized again, and that portion gets a different data materialization status, that data interval is split into multiple data intervals

When a user selects a feature window, multiple data intervals, with different data materialization statuses, might appear in that window. Additionally, multiple data intervals, that are disjointed on the timeline, might also appear. For example, the earlier snapshot has 16 *data intervals* for the defined *feature window* in the offline materialization store.

At any given time, a feature set can have at most 2,000 *data intervals*. Once a feature set reaches that limit, no more materialization jobs can run. Users must then create a new feature set version, with materialization enabled. For the new feature set version, materialize the features in the offline and online stores from scratch.

To avoid the limit, users should run backfill jobs in advance to fill the gaps in the data intervals. This merges the data intervals, and reduces the total count.

# Data materialization jobs

> **Important**
>
> Feature store materialization jobs use Apache Spark. Azure Synapse Runtime for Apache Spark 3.3 reached end of support on March 31, 2025. Use Apache Spark 3.4 or later for your materialization jobs.

Before you run a data materialization job, enable the offline and/or online data materializations at the feature set level.

**Python**

```
from azure.ai.ml.entities import (
    MaterializationSettings,
    MaterializationComputeResource,
)

# Turn on both offline and online materialization on the "accounts" featureset.

accounts_fset_config = fs_client._featuresets.get(name="accounts", version="1")

accounts_fset_config.materialization_settings = MaterializationSettings(
    offline_enabled=True,
    online_enabled=True,
    resource=MaterializationComputeResource(instance_type="standard_e8s_v3"),
    spark_configuration={
        "spark.driver.cores": 4,
        "spark.driver.memory": "36g",
        "spark.executor.cores": 4,
        "spark.executor.memory": "36g",
        "spark.executor.instances": 2,
    },
    schedule=None,
)

fs_poller = fs_client.feature_sets.begin_create_or_update(accounts_fset_config)
print(fs_poller.result())
```

You can submit the data materialization jobs as:

- a backfill job - a manually submitted batch materialization job

- a recurrent materialization job - an automatic materialization job triggered on a scheduled interval.

> **⚠ Warning**
>
> Data already materialized in the offline and/or online materialization will no longer be usable if offline and/or online data materialization is disabled at the feature set level. The data materialization status in offline and/or online materialization store will be reset to `None`.

You can submit backfill jobs by:

- Data materialization status
- The job ID of a canceled or failed materialization job

## Data backfill by data materialization status

Users can submit a backfill request with:

- A list of data materialization status values - Incomplete, Complete, or None
- A feature window (optional)

**Python**

```
from datetime import datetime
from azure.ai.ml.entities import DataAvailabilityStatus

st = datetime(2022, 1, 1, 0, 0, 0, 0)
et = datetime(2023, 6, 30, 0, 0, 0, 0)

poller = fs_client.feature_sets.begin_backfill(
    name="transactions",
    version="1",
    feature_window_start_time=st,
    feature_window_end_time=et,
    data_status=[DataAvailabilityStatus.NONE],
)
print(poller.result().job_ids)
```

After submission of the backfill request, a new materialization job is created for each *data interval* that has a matching data materialization status (Incomplete, Complete, or None). Additionally,

the relevant data intervals must fall within the defined *feature window*. If the data materialization status is `Pending` for a *data interval*, no materialization job is submitted for that interval.

Both the start time and end time of the feature window are optional in the backfill request:

- If the *feature window* start time isn't provided, the start time is defined as the start time of the first *data interval* that doesn't have a data materialization status of `None`.
- If the *feature window* end time isn't provided, the end time is defined as the end time of the last *data interval* that doesn't have a data materialization status of `None`.

> **Note**
>
> If no backfill or recurrent jobs have been submitted for a feature set, the first backfill job must be submitted with a *feature window* start time and end time.

This example has these current data interval and materialization status values:

Expand table

| Start time | End time | Data materialization status |
|---|---|---|
| 2025-04-01T04:00:00.000 | 2025-04-02T04:00:00.000 | None |
| 2025-04-02T04:00:00.000 | 2025-04-03T04:00:00.000 | Incomplete |
| 2025-04-03T04:00:00.000 | 2025-04-04T04:00:00.000 | None |
| 2025-04-04T04:00:00.000 | 2025-04-05T04:00:00.000 | Complete |
| 2025-04-05T04:00:00.000 | 2025-04-06T04:00:00.000 | None |

This backfill request has these values:

- Data materialization `data_status=[DataAvailabilityStatus.Complete, DataAvailabilityStatus.Incomplete]`
- Feature window start = `2025-04-02T12:00:00.000`
- Feature window end = `2025-04-04T12:00:00.000`

It creates these materialization jobs:

- Job 1: process feature window [`2025-04-02T12:00:00.000`, `2025-04-03T04:00:00.000`)

- Job 2: process feature window [`2025-04-04T04:00:00.000`, `2025-04-04T12:00:00.000`)

If both jobs complete successfully, the new data interval and materialization status values become:

Expand table

| Start time | End time | Data materialization status |
|---|---|---|
| 2025-04-01T04:00:00.000 | 2025-04-02T04:00:00.000 | None |
| 2025-04-02T04:00:00.000 | 2025-04-02T12:00:00.000 | Incomplete |
| 2025-04-02T12:00:00.000 | 2025-04-03T04:00:00.000 | Complete |
| 2025-04-03T04:00:00.000 | 2025-04-04T04:00:00.000 | None |
| 2025-04-04T04:00:00.000 | 2025-04-05T04:00:00.000 | Complete |
| 2025-04-05T04:00:00.000 | 2025-04-06T04:00:00.000 | None |

One new data interval is created on day *2025-04-02*, because half of that day now has a different materialization status: `Complete`. Although a new materialization job ran for half of the day *2025-04-04*, the data interval isn't changed (split) because the materialization status didn't change.

If the user makes a backfill request with only data materialization `data_status=[DataAvailabilityStatus.Complete, DataAvailabilityStatus.Incomplete]`, without setting the feature window start and end time, the request uses the default value of those parameters mentioned earlier in this section, and creates these jobs:

- Job 1: process feature window [`2025-04-02T04:00:00.000`, `2025-04-03T04:00:00.000`)
- Job 2: process feature window [`2025-04-04T04:00:00.000`, `2025-04-05T04:00:00.000`)

Compare the feature window for these latest request jobs, and the request jobs shown in the previous example.

## Data backfill by job ID

A backfill request can also be created with a job ID. This is a convenient way to retry a failed or canceled materialization job. First, find the job ID of the job to retry:

- Navigate to the feature set **Materialization jobs** UI

- Select the **Display name** of a specific job that has a *Failed* **Status** value
- At the job **Overview** page, locate the relevant job ID value under the **Name** property It starts with `Featurestore-Materialization-`.

SDK

Python

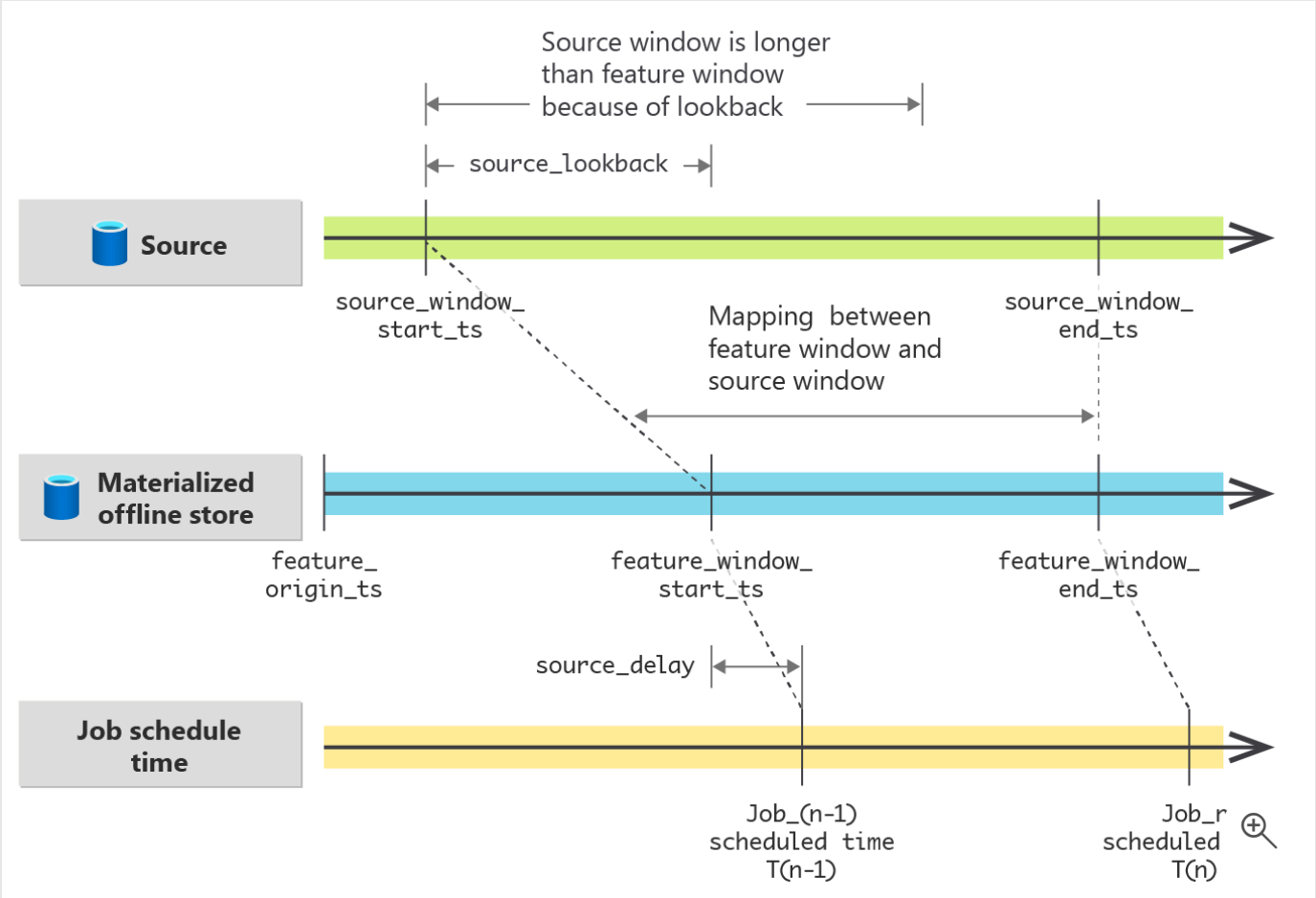
```
poller = fs_client.feature_sets.begin_backfill(
    name="transactions",
    version=version,
    job_id="<JOB_ID_OF_FAILED_MATERIALIZATION_JOB>",
)
print(poller.result().job_ids)
```

You can submit a backfill job with the job ID of a failed or canceled materialization job. In this case, the *feature window* data status for the original failed or canceled materialization job should be `Incomplete`. If this condition isn't met, the backfill job by ID results in a user error. For example, a failed materialization job might have a *feature window* start time `2025-04-01T04:00:00.000` value, and an end time `2025-04-09T04:00:00.000` value. A backfill job submitted using the ID of this failed job succeeds only if the data status everywhere, in the time range `2025-04-01T04:00:00.000` to `2025-04-09T04:00:00.000`, is `Incomplete`.

# Guidance and best practices

## Set proper `source_delay` and recurrent schedule

The `source_delay` property for the source data indicates the delay between the acquisition time of consumption-ready data, compared to the event time of data generation. An event that happened at time `t` lands in the source data table at time `t + x`, because of the upstream data pipeline latency. The `x` value is the source delay.




For proper set-up, the recurrent materialization job schedule accounts for latency. The recurrent job produces features for the `[schedule_trigger_time - source_delay - schedule_interval, schedule_trigger_time - source_delay)` time window.

YAML

```yaml
materialization_settings:
  schedule:
    type: recurrence
    interval: 1
    frequency: Day
    start_time: "2025-04-15T04:00:00.000"
```

This example defines a daily job that triggers at 4 AM, starting on 4/15/2025. Depending on the `source_delay` setting, the job run of 5/1/2025 produces features in different time windows:

- `source_delay=0` produces feature values in window `[2025-04-30T04:00:00.000, 2025-05-01T04:00:00.000)`
- `source_delay=2hours` produces feature values in window `[2025-04-30T02:00:00.000, 2025-05-01T02:00:00.000)`

- `source_delay=4hours` produces feature values in window `[2025-04-30T00:00:00.000, 2025-05-01T00:00:00.000)`

## Update materialization store

Before you update an online feature store or an offline materialization store, all feature sets in that feature store should have the corresponding offline and/or online materialization disabled. The update operation fails as `UserError` if some feature sets have materialization enabled.

The materialization status of the data in the offline and/or online materialization store resets if offline and/or online materialization is disabled on a feature set. The reset renders materialized data unusable. If offline and/or online materialization on the feature set is enabled later, users must resubmit their materialization jobs.

## Online data bootstrap

Online data bootstrap is only applicable if submitted offline materialization jobs have successfully completed. If only offline materialization was initially enabled for a feature set, and online materialization is enabled later, then:

- The default data materialization status of the data in the online store is `None`
- When an online materialization job is submitted, the data with `Complete` materialization status in the offline store is used to calculate online features. This is called online data bootstrapping. Online data bootstrapping saves computational cost because it reuses already-computed features saved in the offline materialization store This table summarizes the offline and online data status values in data intervals that would result in online data bootstrapping:

Expand table

| Start time | End time | Offline data status | Online data status | Online data bootstrap |
|---|---|---|---|---|
| `2025-04-01T04:00:00.000` | `2025-04-02T04:00:00.000` | `None` | `None` | No |
| `2025-04-02T04:00:00.000` | `2025-04-03T04:00:00.000` | `Incomplete` | `None` | No |

| Start time | End time | Offline data status | Online data status | Online data bootstrap |
|---|---|---|---|---|
| 2025-04-03T04:00:00.000 | 2025-04-04T04:00:00.000 | Pending | None | No materialization job submitted |
| 2025-04-04T04:00:00.000 | 2025-04-05T04:00:00.000 | Complete | None | Yes |

## Address source data errors and modifications

Some scenarios modify the source data because of an error, or other reasons, after the data materialization. In these cases, a feature data refresh, for a specific feature window across multiple data intervals, can resolve erroneous or stale feature data. Submit the materialization request for erroneous or stale feature data resolution in the feature window, for the data statuses `None`, `Complete`, and `Incomplete`.

You should submit a materialization request for a feature data refresh only when the feature window doesn't contain any data interval with a `Pending` data status.

## Filling the gaps

In the materialization store, the materialized data might have gaps because:

- a materialization job was never submitted for the feature window
- materialization jobs submitted for the feature window failed, or were canceled

In this case, submit a materialization request in the feature window for `data_status=[DataAvailabilityStatus.NONE,DataAvailabilityStatus.Incomplete]` to fill the gaps. A single materialization request fills all the gaps in the feature window.

# Next steps

- Tutorial 1: Develop and register a feature set with managed feature store
- GitHub Sample Repository

Last updated on 01/30/2026